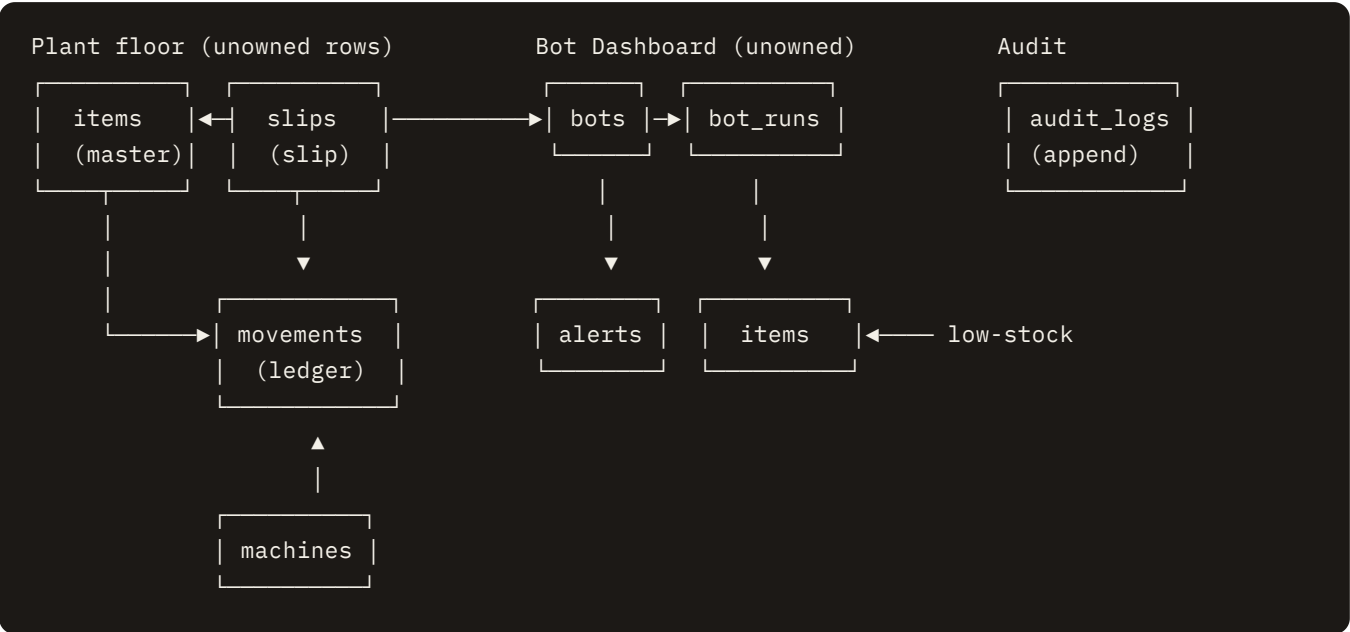




Database Design — Procurement Hub & Bot Dashboard

Single source of truth: `migrations/*.sql` (applied to Neon on deploy via `npm run build` and to PGlite preview via `src/lib/db.ts import.meta.glob`). Auth tables live in `migrations/auth/0001_auth.sql` and are only copied to `migrations/0001_auth.sql` when `VITE_AUTH_ENABLED=true`.

Overview



All tables are **unowned** (no `user_id`) — one shared plant. When auth is turned on, add `user_id TEXT NOT NULL` and scope every query with `requireUserId()` (`src/lib/auth/verify.server.ts`).

Tables

`items` — `migrations/0002_plant.sql`

Master of stocked spares.

COLUMN	TYPE	NOTES
<code>id</code>	<code>serial PK</code>	

COLUMN	TYPE	NOTES
code	text UNIQUE	e.g. HYD-040
name	text	Hydraulic Oil ISO 68
uom	text	Pcs / Ltr / Kg / Pair
reorder_level	int	Low threshold
qty	int	On-hand (mutated only via decideSlip / receiveSlip)
created_at	timestampz	now()

Indexes: code UNIQUE

machines — migrations/0002_plant.sql

COLUMN	TYPE	NOTES
id	serial PK	
code	text UNIQUE	CNC-01
name	text	CNC Lathe 01
line	text	Machine shop / Press bay

Seed: 6 machines (CNC-01, CNC-02, PRS-04, CNV-A, WLD-2, CMP-R).

slips — migrations/0002_plant.sql

Digital requisition. Replaces paper slip.

COLUMN	TYPE	NOTES
id	serial PK	
token	text UNIQUE	SLIP-2609-0001 (YYMM + seq)
item_id	int FK → items.id	NOT NULL
machine_id	int FK → machines.id	nullable
qty	int	requested
issued_qty	int	default 0
department	text	Production / Maintenance ...
station	text	Lathe cell

COLUMN	TYPE	NOTES
hod_title	text	Production HOD
hod_confirmed	bool	must be true
slip_date	date	
status	text	pending issued partial pr_open received
note	text	store note
created_at	timestampz	
decided_at	timestampz	nullable

Index: slips_status_idx (status, created_at desc)

movements — migrations/0002_plant.sql

Immutable ledger. **Negative qty = issue, positive = receive.**

COLUMN	TYPE	NOTES
id	serial PK	
item_id	int FK → items.id	
machine_id	int FK → machines.id	nullable
slip_id	int FK → slips.id	nullable (history seeds have no slip)
qty	int	-want for issue
kind	text	issue receive
created_at	timestampz	

Indexes: movements_item_idx (item_id, created_at desc), movements_machine_idx (machine_id, created_at desc)

Seed: 36 history movements over 58 days (issues & receives) to power 30d & 8-week charts.

bots — migrations/0003_bot_dashboard.sql

RPA definitions.

COLUMN	TYPE	NOTES
id	serial PK	

COLUMN	TYPE	NOTES
code	text UNIQUE	BOT-INV-SYNC
name	text	
description	text	
type	text	inventory slip reorder report generic
status	text	active paused error draft
cron_expr	text	* / 10 * * * * (nullable)
config	jsonb	per-bot settings, default {}
last_run_at	timestampz	
next_run_at	timestampz	
created_at	timestampz	
updated_at	timestampz	

Indexes: bots_status_idx , bots_type_idx **Seed:** 4 bots (Inventory Sync, Slip Processor, Reorder Watcher, Nightly Report).

bot_runs — migrations/0003_bot_dashboard.sql

Execution history.

COLUMN	TYPE	NOTES
id	serial PK	
bot_id	int FK → bots.id	ON DELETE CASCADE
status	text	running success failed skipped
trigger	text	schedule manual api webhook
started_at	timestampz	
finished_at	timestampz	
duration_ms	int	
input	jsonb	
output	jsonb	

COLUMN	TYPE	NOTES
error	text	nullable
created_at	timestamptz	

Indexes: bot_runs_bot_idx (bot_id, started_at desc) , bot_runs_status_idx **Seed:** 6×4 = 24 runs over last 7 days.

alerts — migrations/0003_bot_dashboard.sql

COLUMN	TYPE	NOTES
id	serial PK	
severity	text	info warn critical
title	text	
message	text	
bot_id	int FK → bots.id	nullable, SET NULL
item_id	int FK → items.id	nullable, SET NULL
acknowledged	bool	default false
created_at	timestamptz	

Indexes: alerts_severity_idx , alerts_ack_idx , alerts_bot_idx , alerts_item_idx **Seed:** Auto-seeded on first GET /api/alerts after items exist — one per item where qty <= reorder_level .

audit_logs — migrations/0003_bot_dashboard.sql

Append-only trail.

COLUMN	TYPE	NOTES
id	serial PK	
actor	text	system (or user email when auth on)
action	text	bot.create / slip.decide / alert.ack ...
entity_type	text	
entity_id	text	
payload	jsonb	

COLUMN	TYPE	NOTES
<code>created_at</code>	<code>timestampz</code>	

Indexes: `audit_logs_entity_idx` , `audit_logs_created_idx`

Auth (optional) — `migrations/auth/0001_auth.sql`

Better Auth `user` , `session` , `account` , `verification` (camelCase, quoted). Only active when copied to `migrations/0001_auth.sql` and `VITE_AUTH_ENABLED != "false"` .

Relationships

- `slips.item_id` → `items.id` (every slip is for one item)
- `slips.machine_id` → `machines.id` (which machine needs it)
- `movements.item_id` → `items.id` , `movements.machine_id` → `machines.id` , `movements.slip_id` → `slips.id`
- `bot_runs.bot_id` → `bots.id` (cascade)
- `alerts.bot_id` → `bots.id` (set null), `alerts.item_id` → `items.id` (set null)
- `audit_logs` is denormalized (no FK) for durability

Computed Views (not tables, via SQL)

- **InventoryRow:** `items` + 30d aggregates from `movements` + weekly buckets + `daysCover = qty / (consumed30/30)`
- **MachineStat:** per-machine `consumed30` , `distinctItems` , `topItem` , breakdown rows
- **Refill Dashboard:** `consumed30` vs `received30` , `receipts` , weekly series

Conventions

- **snake_case** in SQL, **camelCase** in TS (`plant.ts` / `bot.ts` mappers).
- **One migration file = one truth.** Never edit a shipped migration; add `0004_*.sql` .
- **PGLite parity:** `src/lib/db.ts` normalizes `int8→number` , `date→YYYY-MM-DD` , `interval→text` so preview == Neon.
- **Seed idempotency:** `ON CONFLICT (code) DO NOTHING` for bots/machines/items.
- **Indexes** on every FK + status + time for dashboard pagination.

How to add a new table

1. Create `migrations/0004_my_feature.sql` with `create table if not exists ...`
2. Run `npm run build` (also runs `db:migrate` for Neon) or just restart dev (PGLite auto-applies on HMR).
3. Add types in `src/lib/my_feature.ts` , server fns in `src/lib/my_feature-server.ts` , and REST in `src/routes/api/...` + `server/routes/api/...` .

4. Document here and in `/system` ER.

ER Diagram (Mermaid)

```
erDiagram
    items ||--o{ slips : "item_id"
    machines ||--o{ slips : "machine_id"
    items ||--o{ movements : "item_id"
    machines ||--o{ movements : "machine_id"
    slips ||--o{ movements : "slip_id"
    items ||--o{ alerts : "item_id"
    bots ||--o{ bot_runs : "bot_id"
    bots ||--o{ alerts : "bot_id"

    items {
        serial id PK
        text code UK
        text name
        text uom
        int reorder_level
        int qty
    }
    machines {
        serial id PK
        text code UK
        text name
        text line
    }
    slips {
        serial id PK
        text token UK
        int item_id FK
        int machine_id FK
        int qty
        int issued_qty
        text status
    }
    movements {
        serial id PK
        int item_id FK
        int qty
        text kind
    }
    bots {
        serial id PK
        text code UK
        text status
        text cron_expr
        jsonb config
    }
    bot_runs {
        serial id PK
        int bot_id FK
    }
```

```
    text status
    text trigger
  }
  alerts {
    serial id PK
    text severity
    bool acknowledged
  }
```

API Endpoints — Procurement Hub & Bot Dashboard

Two surfaces, one DB (`getSql()`):

- **Type-safe RPC** (`createServerFn` in `src/lib/plant-server.ts` & `src/lib/bot-server.ts`) — used by React via `useQuery` / `useMutation` .
- **Plain REST** (`src/routes/api/*` for dev + `server/routes/api/*` for prod via Nitro) — for curl, RPA, n8n, external bots. OpenAPI at `GET /api/openapi.json` , interactive explorer at `/system` → **API Endpoints** tab.

Base URL (dev): `http://localhost:8080` — all endpoints are JSON, no auth in this deployment (`VITE_AUTH_ENABLED=false`). When auth is enabled, add `Authorization: Bearer <token>` or cookie and scope by `user_id` .

Health

`GET /api/health`

Service + DB driver.

Response 200

```
{ "ok": true, "service": "procurement-hub", "version": "1.0.0", "timestamp": "...", "database": "pglite" }
```

cURL

```
curl http://localhost:8080/api/health | jq
```

Catalog

`GET /api/catalog`

Item & machine master (seeded on first call).

Response

```
{
  "items": [ { "id": 1, "code": "HYD-040", "name": "Hydraulic Oil ISO 68", "uom": "Ltr", "reorderLevel":
```

```
"machines": [{ "id":1, "code":"CNC-01", "name":"CNC Lathe 01", "line":"Machine shop"} ]
}
```

ServerFn: `getCatalog()` (GET)

Inventory

GET `/api/inventory`

Rack with 30d consumption, received, `daysCover` , 8-week sparkline.

Row

```
{ "id":1, "code":"HYD-040", "name":"Hydraulic Oil ISO 68", "uom":"Ltr", "reorderLevel":20, "qty":4, "
```

ServerFn: `getInventory()` (GET)

Machines

GET `/api/machines`

Per-machine consumption & breakdown.

Response

```
[{ "id":1, "code":"CNC-01", "name":"CNC Lathe 01", "line":"Machine shop", "consumed30":19, "distinct"
```

ServerFn: `getMachineStats()` (GET)

Refill

GET `/api/refill`

Refill dashboard — per-item + weekly series.

Response

```
{ "rows": [{ "itemId":1, "name":"Hydraulic Oil ISO 68", "code":"HYD-040", "qty":4, "reorderLevel":20, "
```

ServerFn: `getRefillDashboard()` (GET)

Slips

GET /api/slips + **GET /api/slips?status=pending**

List slips. Filter by `status=pending|issued|partial|pr_open|received`.

cURL

```
curl "http://localhost:8080/api/slips?status=pending" | jq
curl http://localhost:8080/api/slips | jq '[0]'
```

ServerFn: `listSlips()` (GET)

POST /api/slips

Raise a slip. Requires `hodConfirmed: true`.

Body

```
{ "itemId": 1, "machineId": 1, "qty": 5, "department": "Production", "station": "Lathe cell", "hodTitle":
```

Validation: `itemId`, `machineId`, `qty` (`int >=1`), `hodConfirmed`.

Response 200 — created slip.

cURL

```
curl -X POST http://localhost:8080/api/slips -H 'content-type: application/json' -d '{"itemId":
```

ServerFn: `raiseSlip({data:{...}})` (POST)

POST /api/slips/decide

Store decision for a pending slip.

Body

```
{ "id": 1, "mode": "stock" } // stock | split | pr
```

- `stock` — issue full qty (fails if `onHand < qty`, use `split`)
- `split` — issue `min(qty, onHand)`, open PR for remainder
- `pr` — no issue, open PR

Side effects: atomically decrements `items.qty`, inserts `movements` (`kind=issue`, `qty=-want`), updates `slips` to `issued` / `partial` / `pr_open`.

cURL

```
curl -X POST http://localhost:8080/api/slips/decide -H 'content-type: application/json' -d '{
```

ServerFn: `decideSlip({data:{id, mode}})` (POST)

POST /api/slips/receive

Mark an open PR as received. Increments stock.

Body

```
{ "id": 2 }
```

cURL

```
curl -X POST http://localhost:8080/api/slips/receive -H 'content-type: application/json' -d '{
```

ServerFn: `receiveSlip({data:{id}})` (POST)

POST /api/slips → **GET /api/inventory** loop

After `decide / receive`, `inventory.qty` and `consumed30` reflect immediately (no cache staleness — `staleTime: 4s`).

Bots

GET /api/bots + **GET /api/bots?status=active&type=inventory**

List bots. Filters: `status=active|paused|error|draft`, `type=inventory|slip|reorder|report|generic`.

cURL

```
curl http://localhost:8080/api/bots | jq '.[0]'
```

```
curl "http://localhost:8080/api/bots?status=active" | jq
```

ServerFn: `listBots()` (GET), `getBot({data:{id}})` (GET)

POST /api/bots

Create bot.

Body

```
{ "code": "BOT-TEST", "name": "Test Bot", "description": "demo", "type": "generic", "status": "draft", "c
```

Validation: `code` must match `^[A-Z0-9-]+$.`

cURL

```
curl -X POST http://localhost:8080/api/bots -H 'content-type: application/json' -d '{ "code": "B
```

ServerFn: `createBot({data:{...}})` (POST), `updateBot({data:{id,...}})`, `deleteBot({data:{id}})`

POST `/api/bots/trigger`

Manual run. Fails if `status=paused`.

Body

```
{ "id": 1, "input": {} }
```

Response — created `bot_runs` row with `trigger=manual`, random `duration_ms`.

cURL

```
curl -X POST http://localhost:8080/api/bots/trigger -H 'content-type: application/json' -d '{
```

ServerFn: `triggerBot({data:{id,input}})` (POST)

Runs

GET `/api/runs` + **GET** `/api/runs?botId=1&limit=20`

Bot execution history, newest first.

cURL

```
curl "http://localhost:8080/api/runs?limit=5" | jq
curl "http://localhost:8080/api/runs?botId=1&limit=10" | jq
```

Row

```
{ "id": 1, "botId": 1, "botCode": "BOT-INV-SYNC", "botName": "Inventory Sync Bot", "status": "success", "
```

ServerFn: `listBotRuns({data:{botId,limit}})` (GET), `getBotStats()` (GET) for aggregates.

Alerts

GET `/api/alerts` + filters

List alerts.

Query: `severity=info|warn|critical`, `acknowledged=true|false`, `limit=1..100`.

cURL

```
curl http://localhost:8080/api/alerts | jq
curl "http://localhost:8080/api/alerts?severity=critical" | jq
curl "http://localhost:8080/api/alerts?acknowledged=false&limit=10" | jq
```

ServerFn: `listAlerts({data:{...}})` (GET)

POST /api/alerts

Create alert.

Body

```
{"severity":"warn","title":"Low stock: Hydraulic Oil","message":"Qty 4 vs reorder 20","botId":
```

ServerFn: `createAlert({data:{...}})` (POST)

POST /api/alerts/acknowledge

Acknowledge.

Body

```
{"id":1}
```

cURL

```
curl -X POST http://localhost:8080/api/alerts/acknowledge -H 'content-type: application/json'
```

ServerFn: `acknowledgeAlert({data:{id}})` (POST)

OpenAPI

GET /api/openapi.json

OpenAPI 3.0 spec for all endpoints. Also served as `server/routes/api/openapi.json.ts` (prod) and `src/routes/api/openapi[.]json.ts` (dev).

cURL

```
curl http://localhost:8080/api/openapi.json | jq '.info'
```

Interactive docs: `/system` → **API Endpoints** tab has Try-it + curl copy.

Server Functions (RPC) vs REST

USE	CALL	TRANSPORT
React loaders/components	<code>getCatalog()</code> / <code>listSlips()</code> etc.	<code>createServerFn</code> → <code>_serverFn</code> RPC, type-safe, no fetch
External RPA/curl/n8n	<code>GET /api/catalog</code> etc.	<code>src/routes/api/*</code> (dev) + <code>server/routes/api/*</code> (prod) → <code>Response.json</code>

Both share `getSql()` and `ensureSeed()` so seed/migrations are identical.

Error Format

All REST errors are JSON with `error` :

```
{"error":"HOD authorisation required"}
```

Status codes: `400` for validation, `404` for not found, `409` for conflict (e.g., stock moved), `405` for wrong method.

Auth (when enabled)

- Set `VITE_AUTH_ENABLED != "false"` in `.grok/app-env.json`
- Copy `migrations/auth/0001_auth.sql` → `migrations/0001_auth.sql`
- Server functions then use `requireUserId()` to scope queries by `user_id` ; REST handlers check `Authorization: Bearer` via `getRequest()` .

Examples — end-to-end

```
# 1. Raise a slip for Carbide Insert (id 2) on CNC Lathe (id 1)
curl -X POST http://localhost:8080/api/slips -H 'content-type: application/json' -d '{"itemId"
# → SLIP-2609-0007

# 2. Store decides — stock is 0, so PR
curl -X POST http://localhost:8080/api/slips/decide -H 'content-type: application/json' -d '{"
# → pr_open

# 3. Mark PR received (increments stock)
curl -X POST http://localhost:8080/api/slips/receive -H 'content-type: application/json' -d '{"

# 4. Verify inventory updated
curl http://localhost:8080/api/inventory | jq '.[] | select(.code=="CUT-118") | .qty'
```

```
# 5. Trigger a bot
curl -X POST http://localhost:8080/api/bots/trigger -H 'content-type: application/json' -d '{
```